

Plan de Pruebas para Tinder de Mascotas

1. Objetivo y alcance

Objetivo: Validar que la aplicación web "Tinder de Mascotas" cumpla con los requisitos funcionales de negocio (registro de usuarios, gestión de perfiles, ABM de mascotas, sistema de emparejamiento/votos) y los requisitos técnicos (seguridad, persistencia de datos, carga de imágenes), asegurando una experiencia de usuario sin errores.

En alcance:

- **Seguridad:** Registro de usuarios, Login (`/logincheck`), Logout y protección de rutas privadas.
- **Gestión de Usuario:** Creación, edición de perfil y actualización de foto de usuario.
- **Gestión de Mascotas:** Creación, visualización, edición y eliminación (ABM) de mascotas desde `MascotaController`.
- **Interacciones:** Lógica de votación (Me gusta / No me gusta) manejada por `VotoServicio`.
- **Archivos:** Subida y visualización de imágenes (fotos de perfil y mascotas).

Fuera de alcance:

- Pruebas de Carga y Stress (rendimiento extremo del servidor).
- Pruebas en dispositivos móviles nativos.

2. Criterios de entrada y salida

Criterios de Entrada:

- El código debe compilar correctamente sin errores.
- La base de datos MySQL debe estar configurada y accesible.
- Los entornos de desarrollo y prueba deben tener instalada la misma versión de Java (JDK 17).

Criterios de Salida:

- El 100% de los Casos de Prueba críticos (Login, Registro, Crear Mascota) han sido ejecutados y pasaron exitosamente.

3. Estrategia de pruebas

Se aplicará un enfoque de **Pirámide de Pruebas**, validando desde la lógica interna hasta la interfaz gráfica:

1. **Pruebas Unitarias (Nivel de Servicio):**
 - *Herramientas:* JUnit 5 y Mockito.
 - *Objetivo:* Validar la lógica de negocio en aislamiento (ej. `MascotaServicioTest`, `UsuarioServicioTest`). Se simularán (Mock) los repositorios para no tocar la BD real.
2. **Pruebas de Integración (Nivel de Repositorio y Seguridad):**
 - *Herramientas:* `@SpringBootTest`, Spring Security Test.
 - *Objetivo:* Validar que las consultas a MySQL (`UsuarioRepositorio`, `MascotaRepositorio`) funcionen y que las rutas (ej. `/mascota/mis-mascotas`) rechacen usuarios no logueados.
3. **Pruebas Funcionales / E2E (Nivel de Sistema/Aceptación):**
 - *Herramientas:* Selenium WebDriver (referenciado en tu carpeta `e2e` y `pom.xml`).
 - *Objetivo:* Simular a un usuario real abriendo el navegador, llenando el formulario en `registro.html`, logueándose y agregando una mascota.

4. Recursos

- **Equipo Humano:**
 - 1 Desarrollador Backend (encargado de Pruebas Unitarias e Integración).
 - 1 QA Automation (encargado de configurar Selenium y Pruebas E2E).
- **Hardware/Entornos:**
 - *Entorno de Desarrollo (Local):* PC con Windows/Linux, MySQL Server en puerto 3306.
- **Software/Herramientas:**
 - Lenguaje: Java 17.
 - Framework de Pruebas: JUnit 5, Mockito.
 - Pruebas UI: Selenium WebDriver (Chrome/Firefox).
 - Construcción: Maven.
 - IDE: IntelliJ IDEA / Eclipse.

5. Criterios de aceptación (Paso a Producción)

El software se considerará **Apto para Producción / Entrega final** cuando:

1. Un usuario nuevo pueda registrarse, loguearse, crear el perfil de su mascota, votar a otra mascota y desloguearse sin recibir ninguna página de error genérica de Spring (White Label Error).
2. Si ocurre un error (ej. clave repetida incorrecta), el sistema redirige controladamente a `error.html` o muestra el mensaje en color rojo (`<p th:if="{error != null}" ...>`) sin colgar la app.
3. El log de la consola no arroja Excepciones no controladas (`NullPointerException`) durante el uso normal.

6. Roles y responsabilidades

- **Líder de QA:** Aprueba el plan de pruebas y evalúa los resultados finales.
- **Desarrolladores (Backend):** Escriben y mantienen el código de las clases en `src/test/java/.../servicios/`. Deben corregir los bugs reportados.
- **Tester Automator / QA:** Escribe los scripts de Selenium dentro del paquete `com.example.tinder.test.e2e`, ejecuta los reportes generales y audita que Spring Security esté bien configurado en `SecurityConfig.java`.